Perfil técnico, rendimiento y hardware

Documento de apoyo para cotizar un equipo. Explica qué hace el proyecto, qué tipo de carga de cómputo genera, qué se midió en el equipo actual, y qué piezas importan de verdad para acelerarlo.

Fecha de las mediciones: 15 de agosto de 2026. Todas las cifras marcadas como *medido* se obtuvieron en el equipo descrito en la sección 3 y se pueden reproducir con los comandos de la sección 10.

---

1. Qué es el proyecto

Un **simulador de propagación de consenso en redes**, con editor visual de grafos, que corre completamente dentro del navegador. No hay servidor de aplicación: el HTML carga Python compilado a WebAssembly (Pyodide) y toda la lógica —estado, interfaz, y los algoritmos numéricos— está escrita en Python.

El modelo: cada nodo de un grafo tiene un valor y cada arista una probabilidad de transmitir. Los nodos se mandan mensajes al azar y adoptan valores según un algoritmo de consenso. La métrica de interés es la **discrepancia** (`max(valores) - min(valores)`): cuando llega a 0, la red alcanzó consenso.

Hay dos modos de uso, con perfiles de cómputo muy distintos:

	Editor	**Análisis de convergencia (sweep)**
Qué hace	Una ronda sobre un grafo dibujado a mano	Barre la probabilidad `p` de 0 a 1 y grafica la discrepancia media con intervalo de confianza
Escala	1 simulación	Hasta 1001 valores de `p` × 50 000 simulaciones cada uno
Costo	Imperceptible	**Es el que justifica comprar máquina**

El resto del documento habla del **sweep**. El editor no tiene requisitos de hardware dignos de mención.

---

2. Perfil de cómputo

Esta es la sección importante para elegir piezas, porque la carga **no** se parece a lo que la gente suele suponer.

El sweep no hace álgebra lineal (no hay multiplicación de matrices, no usa BLAS). Lo que hace, para cada valor de `p`, es:

1. Generar un tensor de números aleatorios de forma `(S, n, n)` — `S` = simulaciones en paralelo, `n` = número de nodos.
2. Compararlo contra la matriz de pesos para decidir qué aristas transmiten.
3. Reducciones sobre el eje de emisores (`any`, `argmax`, `take_along_axis`).

Es decir: **generación masiva de aleatorios y operaciones elemento a elemento sobre tensores grandes**. Tres consecuencias medidas:

2.1 Corre en un solo hilo

Medido: la razón tiempo-de-CPU / tiempo-de-reloj es **1.00** en todas las configuraciones probadas. NumPy no paraleliza estas operaciones; no hay ninguna llamada a BLAS que pudiera repartir trabajo.

> **Hoy, comprar más núcleos no acelera absolutamente nada.** Solo importa qué tan rápido es *un* núcleo.

Esto cambia con un ajuste de software descrito en la sección 7.

2.2 El 65 % del tiempo es generar números aleatorios

Medido (n=60, S=3000, una ronda de Flip Value):

Operación	Tiempo	%
`fired * rand(S,n,n)` (llaves de selección)	35.7 ms	35.1 %
`np.random.rand(S,n,n)` (sorteo de aristas)	30.5 ms	29.9 %
`argmax(axis=1)`	26.7 ms	26.2 %
comparación, máscara, `any`	9.0 ms	8.8 %

Las dos llamadas al generador aleatorio suman **65 %**. Cualquier mejora importante pasa por ahí.

2.3 No está limitado por el ancho de banda de memoria

Medido: un núcleo alcanza ~6.9 GB/s efectivos, muy por debajo de lo que el chip puede mover. La evidencia de que la memoria no es el techo es que al repartir el trabajo entre procesos el rendimiento **sí** sube (×4, sección 7.1): si el bus estuviera saturado con un núcleo, paralelizar no habría servido.

El ancho de banda sí empieza a pesar al agregar núcleos: la eficiencia del paralelismo cae de 85 % con 2 procesos a 33 % con 12.

---

3. Hardware actual

Medido con `sysctl` y `sw_vers`.

Componente	Valor
Chip	Apple M3 Pro (arm64)
Núcleos	12 totales: **6 de rendimiento + 6 de eficiencia**
Memoria	18 GB unificada
Sistema	macOS 26.5.2
Python	3.12.12 (arm64)
NumPy	2.4.5, con SIMD NEON/ASIMD activo

Es una máquina portátil de gama alta, y sirve perfectamente para desarrollar y para estudios chicos. Se queda corta en dos ejes concretos: núcleos de rendimiento (solo 6) y memoria (18 GB), ambos detallados abajo.

---

4. Rendimiento medido

4.1 Rendimiento bruto por núcleo

Simulaciones-ronda por segundo, un núcleo:

n (nodos)	sim-rondas / s	GB/s efectivos
10	754 000	4.8
30	121 000	6.9
100	10 400	6.6
300	1 110	6.4

n (nodos)	sim-rondas / s	GB/s efectivos
1000	108	6.9

El costo crece con **n²** (el tensor es `S × n × n`): duplicar el número de nodos cuadruplica el trabajo.

4.2 Tiempo de un barrido completo

Configuración típica de la aplicación: 201 valores de `p`, 5000 simulaciones por valor. Un núcleo:

n	1 ronda	10 rondas	RAM del tensor
10	1 s	13 s	< 1 MB
30	8 s	1 min	1 MB
100	2 min	16 min	8 MB
300	15 min	**2.5 h**	72 MB
1000	2.6 h	**1.1 días**	800 MB

4.3 Estudio realista

El caso de uso que se tiene en mente —ver cómo cambia el comportamiento al crecer `n`— requiere un barrido completo por cada valor de `n`:

Estudio	1 núcleo (hoy)	Paralelizado (×4)	+ float32
8 valores de n entre 5 y 40	8 min	2 min	1 min
8 valores de n entre 10 y 300	**4.7 h**	1.2 h	48 min

---

5. Límite de memoria

El tensor `(S, n, n)` en float64 domina el consumo. Tamaño en GB:

n	S=1000	S=5000	S=20000	S=50000
30	0.0	0.0	0.1	0.4
100	0.1	0.4	1.6	4.0
300	0.7	3.6	**14.4**	**36.0**
1000	8.0	**40.0**	**160**	**400**
2000	**32.0**	**160**	**640**	**1600**

En negrita, lo que **no cabe** en los 18 GB actuales dejando espacio al sistema.

Matiz importante: la aplicación divide el trabajo en iteraciones, así que el tensor vivo es `S_por_iteración × n × n`, no el total. Con los valores por omisión (5000 simulaciones en 50 iteraciones = 100 por iteración) el consumo es modesto incluso con n=1000. La memoria se vuelve el límite cuando se usan pocas iteraciones con muchas simulaciones, o si se implementa el barrido en 2D.

Conclusión de memoria: 18 GB alcanzan para $n \leq 300$ con configuraciones normales. Para trabajar cómodamente con $n \geq 500$, o para barridos en 2D, conviene 64 GB.

---

6. Qué priorizar al comprar

En orden de impacto real sobre este proyecto:

6.1 Rendimiento de un solo núcleo — prioridad máxima

Es lo único que acelera el código tal como está hoy. Al comparar procesadores, la métrica relevante es el rendimiento *mononúcleo*, no el número de núcleos ni el rendimiento multinúcleo agregado.

6.2 Núcleos de rendimiento — alta, condicionada

Solo sirven si se implementa el paralelismo (sección 7.1, ~30 líneas de código). Con esa mejora, la aceleración medida en el equipo actual fue **×4**, y se estancó porque solo tiene 6 núcleos rápidos.

Advertencia: cuentan los núcleos **de rendimiento**. Los de eficiencia aportan poco; parte de la caída de eficiencia medida (de 85 % a 33 %) se explica por ellos.

6.3 Memoria — alta

64 GB es el punto dulce. 128 GB solo si se plantean grafos de más de 1000 nodos o barridos en 2D con muchas simulaciones.

Importa también el **ancho de banda**, porque limita cuánto rinde el paralelismo: mientras más núcleos trabajen sobre tensores grandes, más se nota. Preferir memoria de alta frecuencia y el máximo número de canales que soporte la plataforma.

6.4 GPU — opcional, alto techo

Hoy **no se usa en absoluto**. Pero la carga (aleatorios masivos + operaciones elemento a elemento + reducciones) es de las que mejor se trasladan a GPU. Si se implementa el camino descrito en 7.3, es donde está la mejora más grande posible.

Si se compra pensando en eso: NVIDIA con soporte CUDA, y la VRAM acota $S \times n^2$ igual que la RAM en la tabla de la sección 5.

Si no se piensa implementar, cualquier gráfica integrada sirve.

6.5 Almacenamiento — irrelevante

El proyecto no hace entrada/salida durante el cálculo. Un SSD normal sobra.

---

7. Mejoras futuras y su implementación

Ordenadas por relación impacto/esfuerzo.

7.1 Paralelizar el barrido — ×4 medido, ~30 líneas

Qué: el bucle sobre los valores de `p` es *vergonzosamente paralelo*: cada punto es independiente. Repartirlo entre procesos con `concurrent.futures.ProcessPoolExecutor`.

Medido en el equipo actual:

Procesos	Aceleración	Eficiencia
2	1.71×	85 %
4	2.82×	71 %
6	3.59×	60 %

Procesos	Aceleración	Eficiencia
8	3.92×	49 %
10	**4.05×**	41 %
12	3.95×	33 %

Se estanca en ×4 porque hay 6 núcleos rápidos. En una máquina con más núcleos de rendimiento y más ancho de banda, escalaría más.

Dónde: el bucle `for probability_index, probability in enumerate(probabilities)` de `iter_sweep`, en `python/simulation/sweep.py`.

Salvedad: esto aplica a la ejecución **nativa** (scripts de investigación). Dentro del navegador habría que usar varios Web Workers, que es bastante más trabajo.

7.2 Usar float32 — ×1.47 medido, cambio pequeño

Qué: los tensores son de float64 (8 bytes). La simulación no necesita esa precisión: son comparaciones contra números aleatorios uniformes.

Medido: generar y comparar en float32 es **1.47× más rápido** y usa **la mitad de memoria** —lo que además duplica el `n` máximo alcanzable.

Dónde: `who_gets_message_batch` y los algoritmos batched en `python/simulation/algorithms.py`.

Salvedad: cambia los resultados numéricos (otra secuencia aleatoria), así que hay que regrabar la línea base del arnés de equivalencia y verificar que las curvas coincidan estadísticamente.

7.3 Camino GPU — mejora potencialmente grande, esfuerzo medio

Qué: reemplazar NumPy por CuPy (o PyTorch) en el motor batched. La API de CuPy es casi idéntica a la de NumPy, así que el motor podría soportar ambos seleccionando el módulo de arreglos.

Por qué encaja: el 65 % del tiempo se va generando aleatorios, que es justamente lo que una GPU hace masivamente en paralelo. El resto son operaciones elemento a elemento y reducciones, igual de favorables.

Impacto: *no medido* — no hay GPU NVIDIA en este equipo. Cargas de este tipo suelen mejorar entre uno y dos órdenes de magnitud, pero **debe verificarse antes de justificar una compra con ese número**.

Salvedad importante: esto solo sirve para ejecución nativa. La versión del navegador seguiría igual.

7.4 Generadores de topología y barrido en 2D — funcionalidad, no velocidad

Poder crear grafos estándar (ciclo, completa, …) dando solo `n`, y barrer `p` contra un segundo parámetro. Multiplica el costo de cómputo por el número de puntos del segundo eje: es la mejora que más presión pone sobre el hardware.

7.5 Lo que no vale la pena

- **Cambiar el generador aleatorio a PCG64:** *medido*, resultó **0.76×**, o sea más lento que el generador actual. Descartado.
- **Optimizar las partes que no son aleatorios:** entre comparación, máscara y `any` suman 8.8 % del tiempo. El techo de mejora es despreciable.

---

8. Resumen para quien cotiza

Pieza	Criterio	Justificación
CPU	Máximo rendimiento **mononúcleo**; luego, máximo número de núcleos **de rendimiento**	El código es monohilo hoy; con un cambio menor escala ×4 y más
RAM	64 GB, alta frecuencia, máximos canales	El consumo crece con n^2; el ancho de banda limita el paralelismo
GPU	Solo si se implementará el camino CuPy: NVIDIA con CUDA y VRAM amplia	La carga encaja muy bien, pero hoy no se usa
Almacenamiento	SSD estándar	No hay E/S durante el cálculo
Otros	Buena refrigeración	Las corridas son de horas a carga sostenida en pocos núcleos

Dos escenarios de compra:

- **Presupuesto contenido:** priorizar CPU con alto rendimiento mononúcleo y 64 GB de RAM. Sin GPU dedicada. Con las mejoras 7.1 y 7.2 se obtiene del orden de 6× sobre el equipo actual, sin tocar hardware exótico.
- **Presupuesto amplio:** lo anterior con más núcleos de rendimiento, más GPU NVIDIA para habilitar el camino 7.3. Antes de comprometer el gasto en GPU, conviene hacer una prueba de concepto con CuPy y medirla.

---

9. Lo que NO está medido

Por honestidad, para que nadie tome decisiones sobre cifras que no existen:

- **La penalización de Pyodide (navegador) frente a Python nativo.** No se pudo medir porque en este entorno el navegador tiene bloqueado el acceso al servidor local. Todas las cifras de este documento son de **ejecución nativa**. Se suele citar que Pyodide es varias veces más lento que CPython para código NumPy, pero **eso no se verificó aquí** y no debe usarse para justificar una compra. Es fácil de medir: ver sección 10.
- **La aceleración en GPU.** No hay hardware NVIDIA disponible en este equipo.
- **El escalamiento más allá de 12 procesos.** No hay máquina con más núcleos para probarlo; el ×4 medido está limitado por los 6 núcleos de rendimiento.

---

10. Cómo reproducir las mediciones

Desde la raíz del proyecto:

```
python3 -c "import sys; sys.path.insert(0,'python'); import numpy as np, time; from simulation.sweep impo
```

Para medir la penalización del navegador, que es el dato que falta: abrir la aplicación, ejecutar un sweep con parámetros conocidos, cronometrarlo, y comparar contra el mismo cálculo nativo.

---

11. Nota sobre el equipo actual

Conviene separar dos propósitos:

- **Desarrollar y usar la aplicación** (editor, sweeps chicos, demostraciones): el equipo actual sobra. La aplicación está pensada para correr en cualquier navegador moderno, incluso modesto.
- **Correr estudios grandes** (muchos valores de n, grafos de cientos de nodos, barridos en 2D): ahí es donde se justifica la máquina nueva, y donde aplican los criterios de la sección 6.